

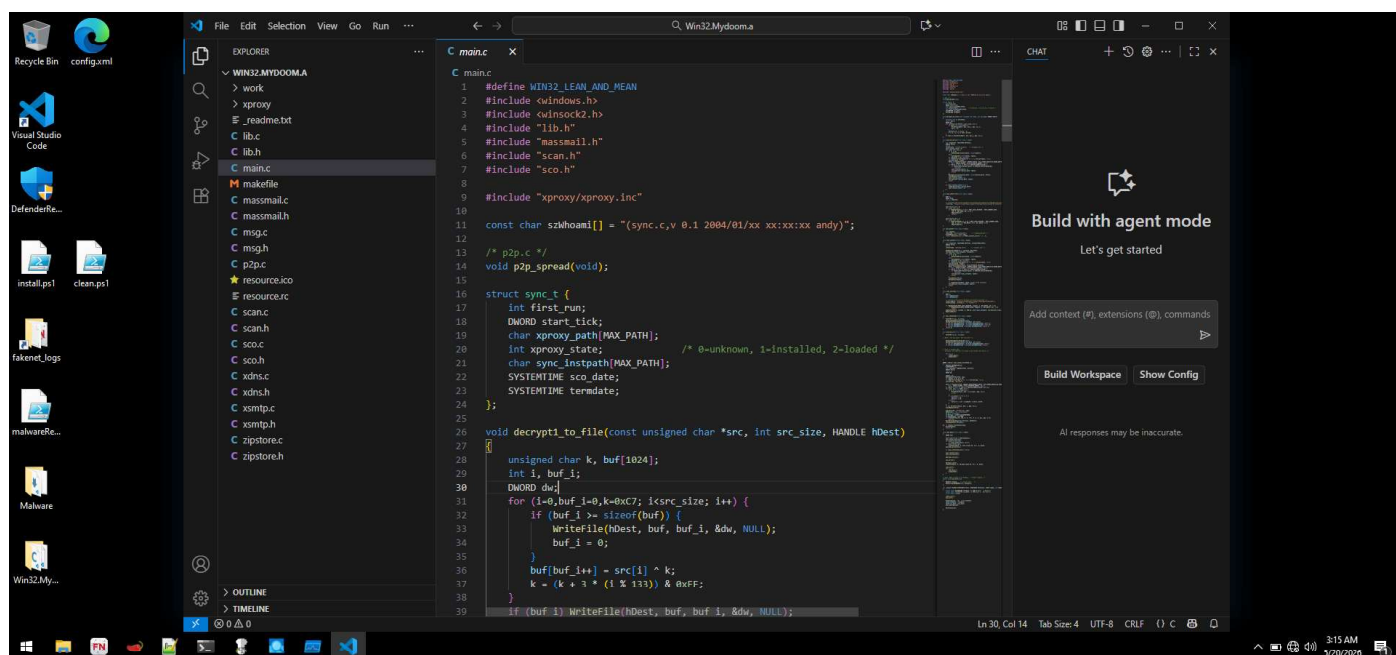
Analisi Forense del Malware Win32.Mydoom.A

Report di Laboratorio – Traccia Extra 1: Sorgente del Malware

Introduzione all'Esercizio

In questo esercizio ho analizzato il codice sorgente originale del worm Win32.Mydoom.A, uno dei malware più distruttivi della storia dell'informatica, apparso per la prima volta nel gennaio 2004. L'obiettivo era duplice: da un lato comprendere in profondità le tecniche utilizzate da questo worm, dall'altro esercitarsi nella risposta rapida a scenari di intelligence, simulando la scoperta di una nuova variante emergente.

Il codice sorgente, disponibile al link fornito nel materiale del corso, è stato scaricato e aperto in un ambiente FlareVM — una macchina virtuale Windows appositamente configurata per l'analisi forense del malware — dove ho esaminato i file .c e .h che compongono l'intero progetto del worm.



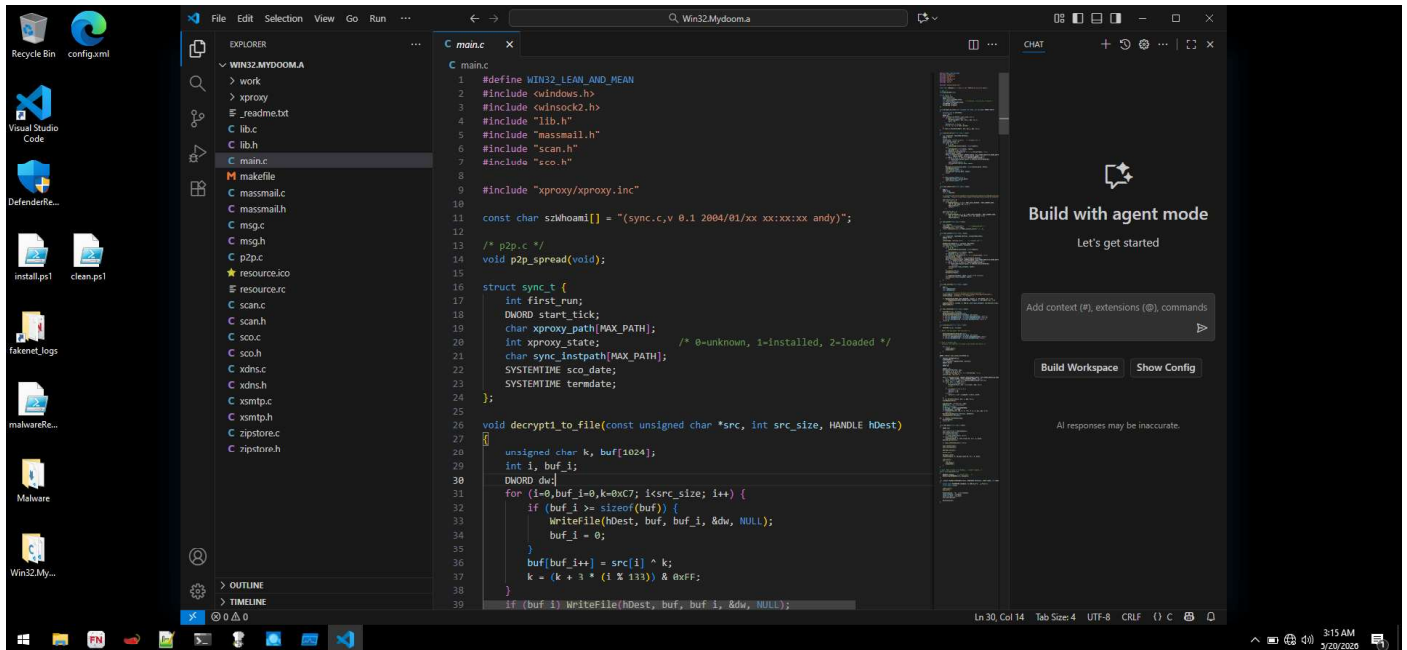
Contesto Storico: Cos'è Mydoom

Mydoom (conosciuto anche come Novarg o Mimail.R) è un worm per sistemi Windows che si è diffuso principalmente tramite allegati e-mail malevoli. All'epoca della sua comparsa riuscì a infettare milioni di macchine in pochi giorni, rallentando sensibilmente il traffico Internet globale. Si stima che nel suo picco di diffusione generasse circa il 25% di tutto il traffico e-mail mondiale.

Le sue finalità principali erano tre: propagarsi il più velocemente possibile tramite e-mail e reti P2P, aprire una backdoor sul sistema infetto lasciando una "porta di servizio" per intrusioni future, e lanciare attacchi DDoS contro bersagli specifici (in particolare il sito di SCO Group).

Struttura del Codice Sorgente

Il progetto è composto da diversi moduli in C, ognuno responsabile di una funzionalità specifica del worm. I file principali includono: `main.c` (entry point e coordinamento generale), `massmail.c/h` (motore di propagazione via e-mail di massa), `scan.c/h` (scansione della rete locale e raccolta di indirizzi), `p2p.c` (diffusione attraverso reti peer-to-peer come KaZaA), `xsmtp.c/h` (client SMTP personalizzato per l'invio diretto delle e-mail), `xdns.c/h` (risoluzione DNS autonoma), `zipstore.c/h` (creazione di allegati ZIP malevoli) e `sco.c/h` (logica per l'attacco DDoS).



Offuscamento tramite ROT13

La prima cosa che ho notato analizzando il codice è che Mydoom non lascia in chiaro nessuna stringa sensibile. Prima ancora di installarsi nel sistema, il worm cerca di nascondere ai software antivirus e agli analisti tutti i nomi di file e le chiavi di registro che utilizzerà, usando una tecnica di offuscamento chiamata ROT13.

ROT13 è una cifratura a sostituzione elementare: ogni lettera dell'alfabeto viene sostituita con quella che si trova 13 posizioni più avanti (e poiché l'alfabeto ha 26 lettere, applicarlo due volte dà il testo originale). Non è crittografia robusta — è banale da decifrare — ma è sufficiente per ingannare una scansione statica veloce che cerca stringhe come "taskmon.exe" o percorsi di registro nel binario.

Nel codice ho trovato queste chiamate significative:

- `rot13(fname, "gnfxzba.rkr")` → decifra in "taskmon.exe", il file con cui il worm si rinomina per sembrare il legittimo Task Monitor di Windows.
- `rot13(valname, "GnfxZba")` → decifra in "TaskMon", il nome usato per la chiave di registro.
- `rot13(fname, "fuvztncv.qyy")` → decifra in "shimgapi.dll", il nome della backdoor installata nel sistema.

1. Tecniche di Offuscamento (Evasione di base)

Prima ancora di installarsi, il malware cerca di nascondere le sue stringhe sensibili agli analisti e agli antivirus per evitare il rilevamento statico immediato.

- **Algoritmo ROT13:** Mydoom usa una semplice cifratura ROT13 (un cifrario a scorrimento di 13 posizioni) per nascondere i nomi dei file e le chiavi di registro.
- Nel codice notiamo:

C

```
rot13(fname, "gnfxzba.rkr");      /* Diventa "taskmon.exe" */
rot13(valname, "GnfxZba");        /* Diventa "TaskMon" */
rot13(fname, "fuvztncv.qyy");    /* Diventa "shimgapi.dll" (il proxy/backdoor) */
```

Tecnica 2: Persistenza e Installazione (sync_install e sync_startup)

Una volta eseguito per la prima volta, il worm deve radicarsi nel sistema per sopravvivere ai riavvii. Questo avviene attraverso due funzioni principali: sync_install e sync_startup.

Copia del file (sync_install)

Il malware usa `GetModuleFileName(NULL, selfpath, MAX_PATH)` per localizzare il proprio eseguibile sul disco (ad esempio in una cartella temporanea, se è stato aperto direttamente dall'allegato e-mail). Poi tenta di copiarsi nella cartella di sistema `system32` o nella cartella dei file temporanei `Temp`, rinominandosi come `taskmon.exe`. Questo nome è scelto strategicamente perché imita il legittimo processo "Task Monitor" di Windows, riducendo la probabilità che un utente poco esperto noti qualcosa di anomalo nel Task Manager.

Chiave di Registro (sync_startup)

Per garantire l'avvio automatico ad ogni accensione del PC, il worm inserisce il percorso del suo eseguibile nella chiave di Autorun del registro di Windows. La chiave target è `Software\Microsoft\Windows\CurrentVersion\Run`, ovviamente anch'essa offuscata nel codice tramite ROT13 come stringa `"Fbsgjner\\Zvpebfbsg\\Jvaqbjf\\PheeragIrefvba\\Eha"`. La funzione utilizza `RegSetValueEx` per scrivere questa voce nel registro, con il valore "TaskMon" che punta al percorso di installazione del worm.

2. Persistenza e installazione (`sync_install` e `sync_startup`)

Queste funzioni mostrano esattamente come il worm si radica nel sistema Windows della vittima.

- **Copia del File (`sync_install`):**

Il malware usa `GetModuleFileName(NULL, selfpath, MAX_PATH);` per trovare dove si trova in quel momento (es. se l'utente lo ha eseguito da un allegato temporaneo). Poi tenta di copiarsi nella cartella di sistema (`System32`) o in quella dei file temporanei (`Temp`) rinominandosi in modo ingannevole come `taskmon.exe` (un nome che imita il legittimo *Task Monitor* di Windows).

- **Chiave di Registro (`sync_startup`):**

Per avviarsi automaticamente a ogni accensione del PC, inserisce il percorso del suo eseguibile modificato nella chiave di *Autorun* di Windows:

C

```
/* Offuscato: "Software\\Microsoft\\Windows\\CurrentVersion\\Run" */
rot13(regpath, "Fbsgjner\\Zvpefbbsg\\Jvaqbjf\\PheeragIrefvba\\Eha");
...
RegSetValueEx(k, valname, 0, REG_SZ, sync->sync_instpath, ...);
```

Tecnica 3: Controllo di Concorrenza con il Mutex

Uno dei meccanismi più interessanti che ho trovato è il controllo della concorrenza tramite mutex. Il problema che questa tecnica risolve è semplice: se il worm si è già installato e sta girando in background, e l'utente apre accidentalmente un altro allegato infetto, non è desiderabile lanciare una seconda istanza del worm — questo raddoppierebbe il carico sulla CPU insospettendo l'utente.

La soluzione adottata dal worm è elegante: al momento dell'avvio, la funzione `sync_mutex` tenta di creare un mutex di sistema con il nome univoco `swebsipcsmtxS0` (anch'esso offuscato in ROT13 come `"FjroFvcpFzgkF0"`). Se il mutex esiste già, la chiamata a `GetLastError()` restituisce `ERROR_ALREADY_EXISTS`, e la funzione ritorna 1, segnalando al codice principale di terminare immediatamente l'esecuzione.

Dal punto di vista della difesa, questo nome del mutex (`swebsipcsmtxS0`) è un ottimo Indicatore di Compromissione (IoC): un analista SOC può cercare la presenza di questo mutex in memoria per rilevare immediatamente un'infezione attiva da Mydoom.

3. Controllo di Concorrenza (Il Mutex)

Per evitare di infettare la stessa macchina più volte (il che sovraccaricherebbe la CPU insospettendo l'utente), Mydoom usa un meccanismo di sincronizzazione:

C

```
int sync_mutex(struct sync_t *sync)
{
    char tmp[64];
    rot13(tmp, "FjroFvcpFzgkF0");      /* Decifrato: "SwebSipcSmtxS0" */
    CreateMutex(NULL, TRUE, tmp);
    return (GetLastError() == ERROR_ALREADY_EXISTS) ? 1 : 0;
}
```

Se il Mutex con il nome unico `SwebSipcSmtxS0` esiste già, significa che il worm è già in esecuzione sul PC, quindi l'istanza corrente si chiude immediatamente (`return;`). Gli Indicatori di Compromissione (IoC) estratti da qui sono cruciali per i difensori SOC.

Il Motore di Propagazione: E-mail di Massa

Il cuore pulsante di Mydoom è il suo motore di propagazione via e-mail, implementato nel modulo `massmail.c`. Il worm non si affida al client di posta della vittima: contiene un client SMTP completo scritto da zero (in `xsmtp.c`) che gli permette di inviare e-mail direttamente ai server di posta dei destinatari, aggirando qualsiasi configurazione locale.

Il processo di propagazione funziona in questo modo: il modulo `scan.c` raccoglie indirizzi e-mail dai file sul disco locale (rubrica, documenti, cache del browser) e li passa al motore di mailing. Ogni e-mail inviata contiene un allegato malevolo con un nome plausibile (come un documento o un'immagine) e il payload cifrato del worm. Quando il destinatario apre l'allegato, il ciclo ricomincia.

La funzione `decrypt1_to_file` visibile nel codice (`main.c`, righe 26-39) decompresse e decifra il payload in memoria prima di scriverlo su disco — un'altra tecnica di evasione per non lasciare il payload in chiaro nel binario. L'algoritmo usa uno XOR con una chiave derivata da una progressione matematica basata sull'indice del byte.

La Backdoor e la Comunicazione C2

Oltre alla propagazione, Mydoom installa una backdoor nel sistema infetto sotto forma di una DLL (`shimgapi.dll`) che apre una porta TCP in ascolto. Questo permette a un attaccante remoto di connettersi alla macchina compromessa, eseguire comandi, scaricare ulteriori payload o usare il sistema come nodo in una botnet per attacchi DDoS.

Il modulo `xproxy.c/inc` implementa questa funzionalità di proxy, permettendo al worm di essere utilizzato come relay per traffico malevolo, rendendo più difficile risalire all'attaccante originale. Il modulo `sco.c` implementa invece la logica per l'attacco DDoS diretto contro il sito di SCO Group, che era il bersaglio specifico di questa variante del worm.

Indicatori di Compromissione (IoC) Estratti dall'Analisi

Dall'analisi del codice sorgente ho estratto i seguenti IoC utili per i difensori SOC:

- **Mutex:** `SwebSipcSmtxS0` — presenza in memoria indica infezione attiva
- **File su disco:** `taskmon.exe` in System32 o %Temp%; `shimgapi.dll` nella directory di sistema
- **Chiave di registro:**
`HKCU\Software\Microsoft\Windows\CurrentVersion\Run\TaskMon`
- **Traffico di rete:** connessioni SMTP in uscita anomale verso porte 25/TCP; porta TCP in ascolto per la backdoor (range 3127-3198)
- **Stringhe offuscate in ROT13:** la presenza di stringhe come "gnfxzba.rkr" o "FjroFvcpFzgkF0" nel binario è un segnale diagnostico preciso

Conclusioni

Questa analisi mi ha permesso di osservare dall'interno come un malware reale orchestri diverse tecniche in modo coordinato: offuscamento per eludere il rilevamento statico, persistenza per sopravvivere ai riavvii, mutex per non esporsi, propagazione massiva per amplificare la portata dell'infezione e backdoor per mantenere l'accesso nel tempo.

Nonostante Mydoom utilizzi tecniche oggi considerate elementari (ROT13, SMTP diretto, XOR semplice), il suo approccio modulare e la logica di auto-preservazione lo hanno reso straordinariamente efficace nel 2004. Studiarlo fornisce una solida base concettuale per comprendere malware moderni molto più sofisticati, che spesso usano le stesse identiche tecniche in forme più evolute.
